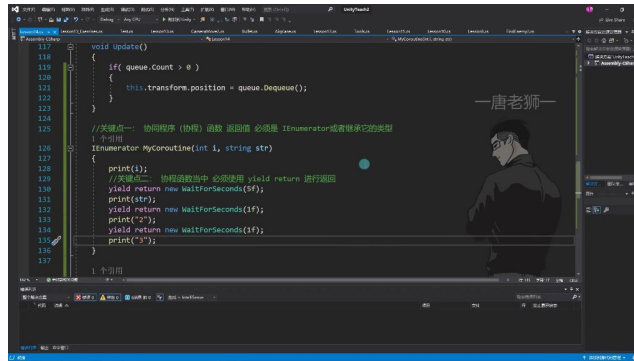
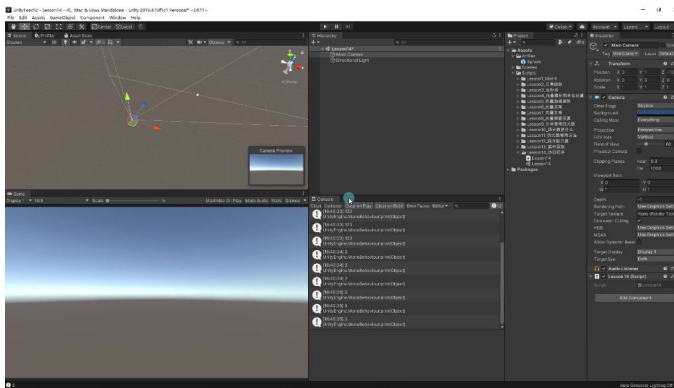




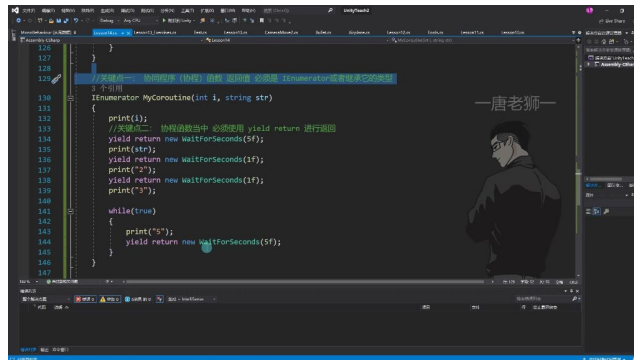


- 执行原理：
  - 遇到yield return会将逻辑分成多部分执行
  - 每个yield return后的内容决定等待时间
  - 示例中分成4部分执行：打印1→等待5秒→打印str→等待1秒→打印"2"→等待1秒→打印"3"

## 2) 关闭携程函数 12:36

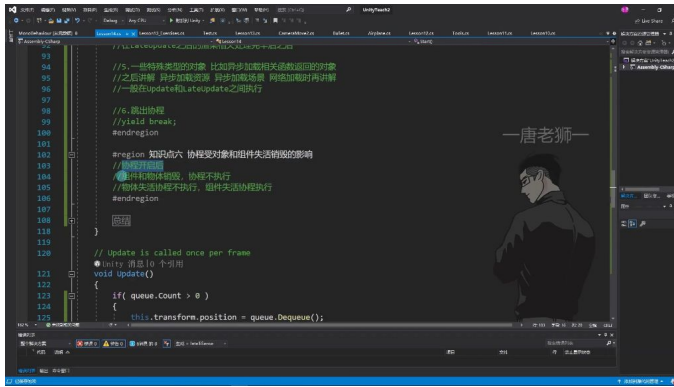


- 关闭所有协程：StopAllCoroutines()
- 关闭指定协程：
  - 通过StartCoroutine返回的Coroutine对象关闭
  - 示例：StopCoroutine(c1)
  - 不推荐使用字符串方式关闭，可能不准确



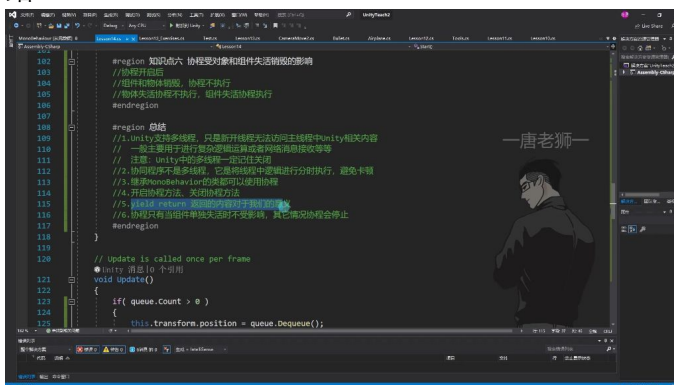
- 特殊用法：
  - 协程内可以写死循环而不会卡死主线程
  - 示例：while(true){ yield return new WaitForSeconds(5f); }
  - 每5秒执行一次循环体内容

## 2. 协程受对象和组件失活销毁的影响 25:06



- 执行条件:
  - 组件和物体销毁时，协程立即停止执行
  - 物体失活时，协程立即停止执行
  - 仅组件失活时，协程仍会继续执行
- 实验验证:
  - 删除挂载协程的GameObject后，协程立即终止
  - 禁用GameObject时，协程输出立即中断
  - 仅禁用脚本组件时，协程仍按原计划每5秒输出一次
- 对比延迟函数:
  - 与Invoke等延迟函数表现一致，说明这是Unity事件系统的统一行为

### 3. 总结 27:52



- 多线程支持:
  - 限制: 新线程不能访问Unity主线程内容
  - 用途: 适合复杂运算和网络通信
  - 注意: 必须手动关闭创建的线程
- 协程本质:
  - 非多线程，采用分时执行逻辑避免卡顿
  - 所有MonoBehaviour派生类均可使用
- 控制方法:
  - 开启: StartCoroutine()
  - 关闭: StopCoroutine()/StopAllCoroutines()
- yield返回值:
  - 不同类型决定不同等待时机
  - 如WaitForSeconds实现定时，WaitForFixedUpdate等待物理帧
- 执行稳定性:
  - 仅组件单独失活不影响执行
  - 其他情况（物体销毁/失活）都会终止协程



- 
- **实践建议:**
  - 使用协程实现计秒器功能
  - 通过协程优化大规模对象生成（如10万个立方体）
  - 注意区分组件失活与物体失活的不同影响

## 二、知识小结

| 知识点                  | 核心内容                                                                                                                                 | 考试重点/<br>易混淆点                 | 难度系数  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|-------|
| <b>Unity<br/>多线程</b> | 支持多线程但新线程无法访问主线程Unity内容，主要用于复杂运算/网络消息接收                                                                                              | <b>必须记住关闭线程</b>               | ★★★★  |
| <b>协同程序概念</b>        | 在主线程内分时分步执行的函数，通过yield return分段，避免卡顿主线程                                                                                              | <b>与线程的本质区别：非并行执行</b>         | ★★★   |
| <b>携程声明规范</b>        | 1. 返回值必须为IEnumerator接口<br>2. 函数内必须包含yield return语句                                                                                   | <b>错误示例：</b><br>直接调用携程函数无效    | ★★★★  |
| <b>携程启停控制</b>        | - 开启：StartCoroutine()<br>- 关闭：StopCoroutine()/StopAllCoroutines()                                                                    | <b>关键区别：</b><br>字符串传参方式不推荐使用  | ★★★   |
| <b>yield返回类型</b>     | 1. null/数字：下一帧执行<br>2. WaitForSeconds：延迟指定秒<br>3. WaitForFixedUpdate：物理帧更新时执行<br>4. WaitForEndOfFrame：渲染完成后执行<br>5. yield break：立即终止 | <b>执行时机差异：</b> 不同返回类型对应不同事件阶段 | ★★★★★ |
| <b>生命周期影响</b>        | - 物体销毁/失活：携程终止<br>- 组件单独失活：携程继续执行                                                                                                    | 与Invoke延迟函数的控制差异              | ★★★★  |
| <b>死循环应用</b>         | 携程内可通过while(true)+yield实现定时循环逻辑                                                                                                      | <b>优势：</b> 不阻塞主线程             | ★★★★  |